

Linked Lists Solved Questions

Every problem from Part 7 of the linked list guide, in order, with a worked C++ solution, the approach behind it, and the mistake that most often breaks it.

33

PROBLEMS

3

TIERS

12

PATTERNS

79

TESTS PASSING

Prepared for **Hari** · on-campus placement preparation

Companion to **linked-lists-cpp-guide.md** · code verified against **reference-all-patterns.cpp**

Every solution in this document was compiled with **g++ -std=c++17 -Wall** and passes its test cases.

Contents

TIER 1 — LEARN THE MECHANICS

1	LC 707	Design Linked List	1 — Traversal & basic ops
2	LC 876	Middle of the Linked List	2 — Fast & slow
3	LC 206	Reverse Linked List	3 — Reversal
4	LC 203	Remove Linked List Elements	4 — Deletion
5	LC 83	Remove Duplicates from Sorted List	4 — Deletion
6	LC 21	Merge Two Sorted Lists	5 — Merge & sort
7	LC 141	Linked List Cycle	6 — Cycle detection
8	LC 160	Intersection of Two Linked Lists	7 — Intersection
9	LC 234	Palindrome Linked List	2 + 3 — combo
10	LC 237	Delete Node in a Linked List	4 — Deletion

TIER 2 — THE PLACEMENT CORE

11	LC 19	Remove Nth Node From End of List	2 — Gap pointers
12	LC 2	Add Two Numbers	9 — Arithmetic
13	LC 142	Linked List Cycle II	6 — Cycle detection
14	LC 92	Reverse Linked List II	3 — Reversal
15	LC 82	Remove Duplicates from Sorted List II	4 — Deletion
16	LC 24	Swap Nodes in Pairs	8 — Rearrangement
17	LC 61	Rotate List	8 — Rearrangement
18	LC 86	Partition List	8 — Rearrangement
19	LC 328	Odd Even Linked List	8 — Rearrangement
20	LC 143	Reorder List	8 — Rearrangement
21	LC 148	Sort List	5 — Merge & sort
22	LC 138	Copy List with Random Pointer	10 — Copy / hashing
23	LC 146	LRU Cache	12 — Design ★

TIER 3 — HARDER, BUT THEY DO APPEAR

24	LC 25	Reverse Nodes in k-Group	3 — Reversal
25	LC 23	Merge k Sorted Lists	5 — Merge & sort
26	LC 445	Add Two Numbers II	9 — Arithmetic
27	LC 430	Flatten a Multilevel Doubly Linked List	12 — Design
28	LC 147	Insertion Sort List	5 — Merge & sort
29	LC 1721	Swapping Nodes in a Linked List	2 — Gap pointers
30	LC 109	Convert Sorted List to Binary Search Tree	Conversion
31	LC 725	Split Linked List in Parts	1 + splitting
32	LC 622	Design Circular Queue	12 — Design
33	LC 460	LFU Cache	12 — Design

Tier 1 — Learn the mechanics

Ten problems. When you can write all of these from a blank page without looking, you have the mechanics. Do not move on before that is true.

01 LC 707 · Design Linked List

MEDIUM Pattern 1 — Traversal & basic ops

PROBLEM

Design your own singly linked list class supporting `get(index)`, `addAtHead(val)`, `addAtTail(val)`, `addAtIndex(index, val)` and `deleteAtIndex(index)`. Indices are 0-based. `get` returns `-1` if the index is invalid; `addAtIndex` does nothing if `index > size`, and appends if `index == size`.

APPROACH

Keep a `head` pointer and a `size` counter — the counter turns every bounds check into $O(1)$ arithmetic instead of a walk. To insert or delete at index i , stop at index $i-1$: you always need the node *before* the one you are touching. Index 0 is the only special case, so route it to `addAtHead`.

Complexity `get` / `addAtTail` / `addAtIndex` / `deleteAtIndex` $O(n)$ · `addAtHead` $O(1)$ · space $O(n)$

C++ SOLUTION

```
class MyLinkedList {
    struct Node {
        int val;
        Node* next;
        Node(int v) : val(v), next(nullptr) {}
    };
    Node* head;
    int size;
public:
    MyLinkedList() : head(nullptr), size(0) {}

    int get(int index) {
        if (index < 0 || index >= size) return -1;
        Node* c = head;
        for (int i = 0; i < index; i++) c = c->next;
        return c->val;
    }

    void addAtHead(int val) {
        Node* n = new Node(val);
        n->next = head;
        head = n;
        size++;
    }

    void addAtTail(int val) {
        Node* n = new Node(val);
        if (!head) head = n;
        else {
            Node* c = head;
            while (c->next) c = c->next;
            c->next = n;
        }
        size++;
    }
};
```

```

}

void addAtIndex(int index, int val) {
    if (index > size) return;           // index == size means append
    if (index <= 0) { addAtHead(val); return; }
    Node* c = head;
    for (int i = 0; i < index - 1; i++) c = c->next;    // stop BEFORE index
    Node* n = new Node(val);
    n->next = c->next;
    c->next = n;
    size++;
}

void deleteAtIndex(int index) {
    if (index < 0 || index >= size) return;
    if (index == 0) {
        Node* d = head; head = head->next; delete d; size--;
        return;
    }
    Node* c = head;
    for (int i = 0; i < index - 1; i++) c = c->next;
    Node* d = c->next;
    c->next = d->next;
    delete d;
    size--;
}
};

```

WATCH OUT

`addAtIndex` must accept `index == size` (append) but reject `index > size`. Keep `size` correct on every path or later bounds checks silently break.

02 LC 876 · Middle of the Linked List

EASY

Pattern 2 — Fast & slow

PROBLEM

Return the middle node of a singly linked list. If there are two middle nodes, return the **second** one.

APPROACH

Move **slow** one step and **fast** two steps per iteration. When **fast** falls off the end, **slow** is exactly halfway. One pass, no length count.

Complexity Time $O(n)$ · Space $O(1)$

C++ SOLUTION

```
ListNode* middleNode(ListNode* head) {
    ListNode *slow = head, *fast = head;
    while (fast && fast->next) {        // order matters: && short-circuits
        slow = slow->next;
        fast = fast->next->next;
    }
    return slow;                        // second middle when length is even
}
```

WATCH OUT

The loop condition must be `fast && fast->next` in that order — `&&` short-circuits left to right, so the first test protects the second. Start `fast = head->next` instead if you want the *first* middle (which is what you need when splitting a list).

03 LC 206 · Reverse Linked List

EASY Pattern 3 — Reversal

PROBLEM

Reverse a singly linked list and return the new head.

APPROACH

Three pointers. Save `next`, flip the current node's arrow to point at `prev`, then slide both `prev` and `curr` forward one node. When `curr` becomes null, `prev` is the new head.

Complexity Time $O(n)$ · Space $O(1)$ — the recursive version is also $O(n)$ time but $O(n)$ stack

C++ SOLUTION

```
ListNode* reverseList(ListNode* head) {
    ListNode* prev = nullptr;
    while (head) {
        ListNode* nxt = head->next; // 1. SAVE the rest
        head->next = prev;           // 2. flip this arrow
        prev = head;                 // 3. prev moves up
        head = nxt;                  // 4. curr moves up
    }
    return prev;                    // prev is the new head
}
```

WATCH OUT

This is the single most important template in the topic — everything from LC 92 to LC 234 to LC 25 is built on it. Line 1 (`nxt = head->next`) is not optional: the instant you write `head->next = prev`, the rest of the list is unreachable.

04 LC 203 · Remove Linked List Elements

EASY Pattern 4 — Deletion

PROBLEM

Remove every node whose value equals `val` and return the head of the resulting list.

APPROACH

The head itself may need deleting, so put a dummy node in front of it. Then walk with a pointer that always sits one node *before* the candidate, and unlink by rewiring `curr->next`.

Complexity Time $O(n)$ · Space $O(1)$

C++ SOLUTION

```
ListNode* removeElements(ListNode* head, int val) {
    ListNode dummy(0);
    dummy.next = head;           // dummy => the head is not a special case
    ListNode* curr = &dummy;
    while (curr->next) {
        if (curr->next->val == val) curr->next = curr->next->next; // do NOT advance
        else curr = curr->next;
    }
    return dummy.next;
}
```

WATCH OUT

After a deletion, do **not** advance `curr` — a new node has just slid into that position and it may also need removing. That is why the code is an `if/else`, not an `if` followed by an unconditional step.

05 LC 83 · Remove Duplicates from Sorted List

EASY Pattern 4 — Deletion

PROBLEM

Given a **sorted** linked list, delete duplicates so that each value appears exactly once. Return the list, still sorted.

APPROACH

Because the list is sorted, equal values are adjacent. Compare each node with its successor; if they match, unlink the successor and stay put, otherwise step forward.

Complexity Time $O(n)$ · Space $O(1)$

C++ SOLUTION

```
ListNode* deleteDuplicates(ListNode* head) {
    ListNode* curr = head;
    while (curr && curr->next) {
        if (curr->val == curr->next->val) curr->next = curr->next->next;
        else curr = curr->next;
    }
    return head;
} // head can never be deleted -> no dummy needed
```

WATCH OUT

No dummy node is needed here — the head keeps one copy of its value, so it can never be deleted. Contrast with LC 82, which removes *every* copy and therefore does need a dummy.

06 LC 21 · Merge Two Sorted Lists

EASY

Pattern 5 — Merge & sort

PROBLEM

Merge two sorted linked lists into one sorted list, splicing together the existing nodes rather than allocating new ones.

APPROACH

Build the result behind a dummy node with a `tail` pointer. Repeatedly append whichever head is smaller. When one list runs out, attach the whole remainder of the other in one assignment.

Complexity Time $O(n + m)$ · Space $O(1)$

C++ SOLUTION

```
ListNode* mergeTwoLists(ListNode* a, ListNode* b) {
    ListNode dummy(0);
    ListNode* tail = &dummy;
    while (a && b) {
        if (a->val <= b->val) { tail->next = a; a = a->next; }
        else { tail->next = b; b = b->next; }
        tail = tail->next;
    }
    tail->next = a ? a : b;           // attach the non-empty remainder
    return dummy.next;
}
```

WATCH OUT

Use `<=` rather than `<` to keep the merge stable. The final `tail->next = a ? a : b` is what saves you from writing a second loop.

07 LC 141 · Linked List Cycle

EASY

Pattern 6 — Cycle detection

PROBLEM

Return `true` if the linked list contains a cycle.

APPROACH

Floyd's tortoise and hare. Slow moves one step, fast moves two. Inside a loop the gap between them shrinks by one each iteration, so fast must eventually land on slow. With no loop, fast falls off the end.

Complexity Time $O(n)$ · Space $O(1)$ — the hash-set version is $O(n)$ space

C++ SOLUTION

```
bool hasCycle(ListNode* head) {
    ListNode *slow = head, *fast = head;
    while (fast && fast->next) {
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast) return true; // compare ADDRESSES, not values
    }
    return false; // fast fell off => no cycle
}
```

WATCH OUT

Compare `slow == fast`, comparing **addresses**. Comparing `slow->val == fast->val` is wrong: two different nodes can hold the same value.

08 LC 160 · Intersection of Two Linked Lists

EASY Pattern 7 — Intersection

PROBLEM

Two linked lists may merge at some node and share every node after it. Return the first shared node, or `nullptr` if they never meet.

APPROACH

The only obstacle is that the lists have different lengths before the junction. Let each pointer, on reaching the end of its own list, restart at the head of the *other* list. Both then travel exactly `lenA + lenB` steps, which cancels the difference and syncs them at the junction.

Complexity Time $O(n + m)$ · Space $O(1)$

C++ SOLUTION

```
ListNode* getIntersectionNode(ListNode* a, ListNode* b) {
    if (!a || !b) return nullptr;
    ListNode *p = a, *q = b;
    while (p != q) {
        p = p ? p->next : b;    // when p runs out, restart it on list b
        q = q ? q->next : a;    // when q runs out, restart it on list a
    }
    return p;                  // junction, or nullptr if they never meet
}
```

WATCH OUT

If the lists never intersect, both pointers become `nullptr` on the same step and `nullptr == nullptr` ends the loop — so the trick terminates correctly without a special case.

09 LC 234 · Palindrome Linked List

EASY Pattern 2 + 3 — combo

PROBLEM

Return `true` if the linked list reads the same forwards and backwards. Aim for $O(n)$ time and $O(1)$ space.

APPROACH

Three moves stacked: find the first middle with fast & slow, reverse the second half, then walk the two halves in step comparing values. Reversing the second half afterwards leaves the caller's list unchanged — mention this in an interview, it is a real point.

Complexity Time $O(n)$ · Space $O(1)$

C++ SOLUTION

```
ListNode* reverseFor234(ListNode* head) {
    ListNode* prev = nullptr;
    while (head) { ListNode* n = head->next; head->next = prev; prev = head; head = n; }
    return prev;
}

bool isPalindrome(ListNode* head) {
    if (!head || !head->next) return true;

    ListNode *slow = head, *fast = head->next;    // first middle
    while (fast && fast->next) { slow = slow->next; fast = fast->next->next; }

    ListNode* second = reverseFor234(slow->next);    // reverse the back half
    slow->next = nullptr;

    ListNode *p = head, *q = second;
    bool ok = true;
    while (p && q) {
        if (p->val != q->val) { ok = false; break; }
        p = p->next; q = q->next;
    }
    slow->next = reverseFor234(second);    // restore the input list
    return ok;
}
```

WATCH OUT

Split at the *first* middle (`fast = head->next`) so the front half is never shorter than the back half; then comparing until either pointer runs out is correct for both odd and even lengths.

10 LC 237 · Delete Node in a Linked List

MEDIUM Pattern 4 — Deletion

PROBLEM

You are given **only** the node to be deleted — not the head. The node is guaranteed not to be the last one. Delete it from the list.

APPROACH

You cannot reach the predecessor, so you cannot unlink this node. Instead, impersonate the successor: copy its value into the current node, then unlink the successor. The list's values end up correct.

Complexity Time $O(1)$ · Space $O(1)$

C++ SOLUTION

```
void deleteNode(ListNode* node) {  
    node->val = node->next->val;    // copy the successor's value into me  
    node->next = node->next->next;  // then unlink the successor  
}
```

WATCH OUT

This works only because the node is never the tail — there has to be a successor to steal from. It is a favourite interview question precisely because it tests whether you understand that a node has no way to reach the one before it.

Tier 2 — The placement core

These are what actually get asked in on-campus drives. Most are combinations of two Tier 1 patterns — name the sub-patterns aloud as you decompose each one.

11 LC 19 · Remove Nth Node From End of List

MEDIUM Pattern 2 — Gap pointers

PROBLEM

Remove the n -th node counting from the end of the list and return the head. Do it in one pass.

APPROACH

Open a gap of exactly n nodes between two pointers, then advance both until the leader reaches the last node. The follower is now sitting on the node *before* the target, which is exactly where you need to be to unlink it.

Complexity Time $O(n)$ · Space $O(1)$

C++ SOLUTION

```
ListNode* removeNthFromEnd(ListNode* head, int n) {
    ListNode dummy(0);
    dummy.next = head;
    ListNode *slow = &dummy, *fast = &dummy;
    for (int i = 0; i < n; i++) fast = fast->next; // open a gap of n
    while (fast->next) { slow = slow->next; fast = fast->next; }
    slow->next = slow->next->next; // slow sits BEFORE the target
    return dummy.next;
}
```

WATCH OUT

Start both pointers at the dummy, not at the head. When `n` equals the list length you are deleting the head, and the dummy is what makes that case need no extra code.

12 LC 2 · Add Two Numbers

MEDIUM Pattern 9 — Arithmetic

PROBLEM

Two non-negative integers are stored as linked lists with digits in **reverse** order (`2→4→3` means 342). Add them and return the sum in the same reversed form.

APPROACH

Reverse order is a gift: the least significant digits line up at the heads, so a single walk with a running carry does the whole job. Build the result behind a dummy node.

Complexity Time $O(\max(n, m))$ · Space $O(\max(n, m))$ for the output

C++ SOLUTION

```
ListNode* addTwoNumbers(ListNode* a, ListNode* b) {
    ListNode dummy(0);
    ListNode* tail = &dummy;
    int carry = 0;
    while (a || b || carry) {           // "|| carry" handles 999 + 1
        int sum = carry;
        if (a) { sum += a->val; a = a->next; }
        if (b) { sum += b->val; b = b->next; }
        carry = sum / 10;
        tail->next = new ListNode(sum % 10);
        tail = tail->next;
    }
    return dummy.next;
}
```

WATCH OUT

The loop condition is `while (a || b || carry)`. Dropping `|| carry` is the classic bug — it silently loses the leading 1 in cases like $999 + 1$.

13 LC 142 · Linked List Cycle II

MEDIUM Pattern 6 — Cycle detection

PROBLEM

Return the node where the cycle begins, or `nullptr` if there is no cycle.

APPROACH

Phase 1 is Floyd's, giving a meeting point inside the loop. Phase 2: reset one pointer to the head and move both one step at a time — they meet at the loop entrance.

Complexity Time $O(n)$ · Space $O(1)$

C++ SOLUTION

```
ListNode* detectCycle(ListNode* head) {
    ListNode *slow = head, *fast = head;
    while (fast && fast->next) {
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast) { // phase 1: they meet inside the loop
            ListNode* p = head; // phase 2: walk both one step at a time
            while (p != slow) { p = p->next; slow = slow->next; }
            return p; // meeting point == loop entrance
        }
    }
    return nullptr;
}
```

WATCH OUT

Be ready to prove phase 2. Let L = head to loop start, C = loop length, and let them meet x steps into the loop. Fast travels twice as far as slow, so $2(L + x) = L + x + kC$, giving $L = kC - x$. Walking L steps from the head and L steps on from the meeting point therefore land on the same node.

14 LC 92 · Reverse Linked List II

MEDIUM Pattern 3 — Reversal

PROBLEM

Reverse the nodes from position *left* to position *right* (1-indexed, inclusive) and return the list. One pass.

APPROACH

Walk to the node just before position *left* and anchor there. Then repeatedly detach the node *after* `curr` and re-insert it at the front of the segment — do this *right* − *left* times and the segment ends up reversed in place.

Complexity Time $O(n)$ · Space $O(1)$

C++ SOLUTION

```
ListNode* reverseBetween(ListNode* head, int left, int right) {
    ListNode dummy(0);
    dummy.next = head;
    ListNode* prev = &dummy;
    for (int i = 1; i < left; i++) prev = prev->next;    // node just before segment

    ListNode* curr = prev->next;
    for (int i = 0; i < right - left; i++) {    // head-insertion, right-left times
        ListNode* nxt = curr->next;
        curr->next = nxt->next;
        nxt->next = prev->next;
        prev->next = nxt;
    }
    return dummy.next;
}
```

WATCH OUT

Positions are 1-indexed, so `for (i = 1; i < left; i++)` lands on the node before the segment. A dummy node covers `left == 1`. Note `curr` never moves — it rides down the list as nodes are pulled out in front of it.

15 LC 82 · Remove Duplicates from Sorted List II

MEDIUM Pattern 4 — Deletion

PROBLEM

In a sorted list, delete **every** node that has a duplicate, keeping only values that appear exactly once.

APPROACH

When a run of equal values starts, skip past the whole run, then point `prev` at whatever follows it. `prev` stays put through that — it is still the last confirmed-unique node.

Complexity Time $O(n)$ · Space $O(1)$

C++ SOLUTION

```
ListNode* deleteAllDuplicates(ListNode* head) {
    ListNode dummy(0);
    dummy.next = head;
    ListNode* prev = &dummy;
    ListNode* curr = head;
    while (curr) {
        if (curr->next && curr->val == curr->next->val) {
            int dup = curr->val;
            while (curr && curr->val == dup) curr = curr->next; // skip the whole run
            prev->next = curr; // prev does NOT move
        } else {
            prev = curr;
            curr = curr->next;
        }
    }
    return dummy.next;
}
```

WATCH OUT

Here the head can be deleted (input `1→1→2`), so a dummy is required — unlike LC 83. Getting these two problems straight in your head is worth the five minutes.

16 LC 24 · Swap Nodes in Pairs

MEDIUM Pattern 8 — Rearrangement

PROBLEM

Swap every two adjacent nodes and return the head. You must swap the nodes themselves, not just their values.

APPROACH

With a dummy in front, each swap is three assignments: `a` jumps over `b`, `b` points back at `a`, and `prev` adopts `b`. Then `prev = a`, because `a` is now the tail of the finished part.

Complexity Time $O(n)$ · Space $O(1)$

C++ SOLUTION

```
ListNode* swapPairs(ListNode* head) {
    ListNode dummy(0);
    dummy.next = head;
    ListNode* prev = &dummy;
    while (prev->next && prev->next->next) {
        ListNode* a = prev->next;
        ListNode* b = a->next;
        a->next = b->next;    // a jumps over b
        b->next = a;          // b points back at a
        prev->next = b;        // prev adopts b
        prev = a;             // a is now the tail of the done part
    }
    return dummy.next;
}
```

WATCH OUT

The loop guard `prev->next && prev->next->next` is what leaves a lone final node untouched when the length is odd.

17 LC 61 · Rotate List

MEDIUM Pattern 8 — Rearrangement

PROBLEM

Rotate the list to the right by k places, where k may be much larger than the length.

APPROACH

Walk to the tail, counting the length n on the way. Close the list into a ring, then step $n - k - 1$ nodes from the head to find the new tail, take its successor as the new head, and break the ring.

Complexity Time $O(n)$ · Space $O(1)$

C++ SOLUTION

```
ListNode* rotateRight(ListNode* head, int k) {
    if (!head || !head->next || k == 0) return head;

    int n = 1;
    ListNode* tail = head;
    while (tail->next) { tail = tail->next; n++; }

    k %= n;                                     // k can be far larger than the length
    if (k == 0) return head;

    tail->next = head;                          // close the ring
    ListNode* newTail = head;
    for (int i = 0; i < n - k - 1; i++) newTail = newTail->next;
    ListNode* newHead = newTail->next;
    newTail->next = nullptr;                    // reopen the ring
    return newHead;
}
```

WATCH OUT

`k %= n` first, then return early if `k == 0` — otherwise you do a full pointless rotation, or worse, walk off the list. And never forget `newTail->next = nullptr`: leave the ring closed and the judge loops forever.

18 LC 86 · Partition List

MEDIUM Pattern 8 — Rearrangement

PROBLEM

Given a value x , reorder the list so every node with value less than x comes before every node with value $\geq x$. The **relative order within each group must be preserved**.

APPROACH

Two dummy nodes, two output lists. Walk the input once, appending each node to whichever list it belongs in, then join the two. Order is preserved for free because you append in input order.

Complexity Time $O(n)$ · Space $O(1)$

C++ SOLUTION

```
ListNode* partitionList(ListNode* head, int x) {
    ListNode lessDummy(0), geDummy(0);           // two output lists
    ListNode *less = &lessDummy, *ge = &geDummy;
    while (head) {
        if (head->val < x) { less->next = head; less = less->next; }
        else { ge->next = head; ge = ge->next; }
        head = head->next;
    }
    ge->next = nullptr;                          // ESSENTIAL: without this you build a cycle
    less->next = geDummy.next;
    return lessDummy.next;
}
```

WATCH OUT

`ge->next = nullptr` before joining. Without it the last $\geq$ node still points back into the original list and you have built a cycle. This exact omission is the most common cause of a TLE on this problem.

19 LC 328 · Odd Even Linked List

MEDIUM Pattern 8 — Rearrangement

PROBLEM

Group all nodes at odd **positions** (not odd values) together, followed by all nodes at even positions, preserving relative order. $O(1)$ space, $O(n)$ time.

APPROACH

Run two pointers that leapfrog each other, unweaving the list into an odd chain and an even chain in a single pass. Save the head of the even chain first, then stitch it onto the tail of the odd chain.

Complexity Time $O(n)$ · Space $O(1)$

C++ SOLUTION

```
ListNode* oddEvenList(ListNode* head) {
    if (!head || !head->next) return head;
    ListNode* odd = head;
    ListNode* even = head->next;
    ListNode* evenHead = even;           // remember where the evens start
    while (even && even->next) {
        odd->next = even->next; odd = odd->next;
        even->next = odd->next; even = even->next;
    }
    odd->next = evenHead;                 // stitch evens onto the odds
    return head;
}
```

WATCH OUT

You must save `evenHead` before the loop — after it, there is no way back to it. The guard `even && even->next` leaves `odd` correctly on the last odd node for both parities.

20 LC 143 · Reorder List

MEDIUM Pattern 8 — Rearrangement

PROBLEM

Reorder $L_0 \rightarrow L_1 \rightarrow \dots \rightarrow L_n$ into $L_0 \rightarrow L_n \rightarrow L_1 \rightarrow L_{n-1} \rightarrow L_2 \rightarrow \dots$, in place, without changing node values.

APPROACH

The classic three-move composite: **(1)** find the first middle and cut, **(2)** reverse the back half, **(3)** weave the two halves together one node at a time. Recognising that decomposition *is* the problem.

Complexity Time $O(n)$ · Space $O(1)$

C++ SOLUTION

```
void reorderList(ListNode* head) {
    if (!head || !head->next) return;

    ListNode *slow = head, *fast = head->next;           // 1. find the first middle
    while (fast && fast->next) { slow = slow->next; fast = fast->next->next; }
    ListNode* second = slow->next;
    slow->next = nullptr;                                  // cut

    ListNode* prev = nullptr;                             // 2. reverse the back half
    while (second) {
        ListNode* nxt = second->next;
        second->next = prev;
        prev = second;
        second = nxt;
    }

    ListNode* first = head;                               // 3. weave the halves
    while (prev) {
        ListNode* n1 = first->next;
        ListNode* n2 = prev->next;
        first->next = prev;
        prev->next = n1;
        first = n1;
        prev = n2;
    }
}
```

WATCH OUT

Split at the first middle so the front half is never shorter than the back — then the weave loop can be driven purely by the reversed half running out.

21 LC 148 · Sort List

MEDIUM Pattern 5 — Merge & sort

PROBLEM

Sort a linked list in $O(n \log n)$ time and, ideally, $O(1)$ extra space.

APPROACH

Merge sort is the natural fit: splitting a list is free and merging needs no random access (which is exactly what quicksort would want and linked lists cannot give). Split at the middle, sort each half recursively, merge.

Complexity Time $O(n \log n)$ · Space $O(\log n)$ recursion stack ($O(1)$ if written bottom-up)

C++ SOLUTION

```
ListNode* mergeForSort(ListNode* a, ListNode* b) {
    ListNode dummy(0);
    ListNode* tail = &dummy;
    while (a && b) {
        if (a->val <= b->val) { tail->next = a; a = a->next; }
        else { tail->next = b; b = b->next; }
        tail = tail->next;
    }
    tail->next = a ? a : b;
    return dummy.next;
}

ListNode* sortList(ListNode* head) {
    if (!head || !head->next) return head;

    // split at the FIRST middle (fast starts one ahead) so both halves are non-empty
    ListNode *slow = head, *fast = head->next;
    while (fast && fast->next) { slow = slow->next; fast = fast->next->next; }
    ListNode* second = slow->next;
    slow->next = nullptr; // cut

    return mergeForSort(sortList(head), sortList(second));
}
```

WATCH OUT

Split with `fast = head->next`, not `fast = head`. On a two-node list the latter leaves the first half empty and you recurse forever. And the cut `slow->next = nullptr` is mandatory.

22 LC 138 · Copy List with Random Pointer

MEDIUM Pattern 10 — Copy / hashing

PROBLEM

Each node has a `next` pointer and a `random` pointer that may target any node or be null. Return a **deep copy**: entirely new nodes, with the same structure.

APPROACH

The problem is mapping each original node to its copy. The easy answer is a hash map (old $\rightarrow$ new, $O(n)$ space). The $O(1)$ -space answer: weave each copy directly behind its original, so the copy of `X` is always `X->next` — then `X->next->random = X->random->next` wires every random pointer. Finally unweave the two lists.

Complexity Time $O(n)$ · Space $O(1)$ for the interleaving method, $O(n)$ for the map

C++ SOLUTION

```
class RandomNode {
public:
    int val;
    RandomNode* next;
    RandomNode* random;
    RandomNode(int v) : val(v), next(nullptr), random(nullptr) {}
};

// O(1) extra space: interleave copies, wire randoms, then unweave.
RandomNode* copyRandomList(RandomNode* head) {
    if (!head) return nullptr;

    // 1. A -> A' -> B -> B' -> ...
    for (RandomNode* c = head; c; c = c->next->next) {
        RandomNode* copy = new RandomNode(c->val);
        copy->next = c->next;
        c->next = copy;
    }

    // 2. the copy of X is always X->next
    for (RandomNode* c = head; c; c = c->next->next)
        if (c->random) c->next->random = c->random->next;

    // 3. separate the two interleaved lists
    RandomNode* newHead = head->next;
    for (RandomNode* c = head; c; c = c->next) {
        RandomNode* copy = c->next;
        c->next = copy->next;
        if (copy->next) copy->next = copy->next->next;
    }
    return newHead;
}
```

WATCH OUT

Mention both approaches in an interview: map first (shows you can solve it), then the interleaving trick. Restoring the original list in step 3 is part of a correct answer, not a nicety.

23 LC 146 · LRU Cache

MEDIUM Pattern 12 — Design ★

PROBLEM

Design a cache with a fixed capacity supporting `get(key)` and `put(key, value)` in **O(1) average time**. When the cache is full, evict the least recently used key. Both `get` and `put` count as a use.

APPROACH

A hash map gives O(1) lookup but no ordering; a doubly linked list gives O(1) removal from the middle but no lookup. Combine them: the map stores key → node, the list keeps nodes in recency order with the most recent at the head. Every touch is remove-then-insert-at-front; eviction is always the node before the tail.

Complexity get O(1) · put O(1) · Space O(capacity)

C++ SOLUTION

```
class LRUCache {
    struct Node {
        int key, val;
        Node *prev, *next;
        Node(int k, int v) : key(k), val(v), prev(nullptr), next(nullptr) {}
    };
    int cap;
    unordered_map<int, Node*> mp;
    Node *head, *tail;           // head side = most recent, tail side = least

    void remove(Node* n) {      // sentinels mean no null checks are needed
        n->prev->next = n->next;
        n->next->prev = n->prev;
    }
    void insertFront(Node* n) {
        n->next = head->next;
        n->prev = head;
        head->next->prev = n;
        head->next = n;
    }
public:
    LRUCache(int capacity) : cap(capacity) {
        head = new Node(0, 0);   // dummy head and dummy tail
        tail = new Node(0, 0);
        head->next = tail;
        tail->prev = head;
    }

    int get(int key) {
        if (!mp.count(key)) return -1;
        Node* n = mp[key];
        remove(n);
        insertFront(n);          // touching it makes it most recent
        return n->val;
    }

    void put(int key, int value) {
        if (mp.count(key)) {
            Node* n = mp[key];
            n->val = value;
            remove(n);
            insertFront(n);
            return;
        }
    }
};
```

```

    }
    if ((int)mp.size() == cap) {    // evict from the tail side
        Node* lru = tail->prev;
        remove(lru);
        mp.erase(lru->key);
        delete lru;
    }
    Node* n = new Node(key, value);
    mp[key] = n;
    insertFront(n);
}
};

```

WATCH OUT

This is the most-asked linked list question in on-campus placements — Amazon, Microsoft, Adobe, Flipkart, Uber. Use dummy nodes at *both* ends: `remove()` and `insertFront()` then need no null checks at all, which is what makes the code short enough to write correctly under pressure. Remember to erase the evicted key from the map, not just from the list.

Tier 3 — Harder, but they do appear

Longer problems and the design round. Reach these only once Tier 2 is solid.

24 LC 25 · Reverse Nodes in k-Group

HARD

Pattern 3 — Reversal

PROBLEM

Reverse the nodes of the list k at a time. If the number of remaining nodes is fewer than k , leave them as they are.

APPROACH

First walk k nodes to check a full group exists — if not, return `head` untouched. Then recurse on the rest *first*, so you get back the already-correct head of everything that follows, and reverse this group directly onto it. The recursive return value doubles as the `prev` seed for the reversal.

Complexity Time $O(n)$ · Space $O(n/k)$ recursion stack ($O(1)$ iteratively)

C++ SOLUTION

```
ListNode* reverseKGroup(ListNode* head, int k) {
    ListNode* node = head;
    for (int i = 0; i < k; i++) {        // are there k nodes left?
        if (!node) return head;        // fewer than k -> leave them as they are
        node = node->next;
    }
    // node points just past this group
    ListNode* prev = reverseKGroup(node, k); // solve the rest first
    ListNode* curr = head;
    for (int i = 0; i < k; i++) {        // reverse this group onto prev
        ListNode* nxt = curr->next;
        curr->next = prev;
        prev = curr;
        curr = nxt;
    }
    return prev;
}
```

WATCH OUT

The check-then-recurse-then-reverse order is what makes this short. Solving the group first and stitching afterwards needs far more bookkeeping. Mention the iterative $O(1)$ -space version exists.

25 LC 23 · Merge k Sorted Lists

HARD

Pattern 5 — Merge & sort

PROBLEM

Merge k sorted linked lists into one sorted list.

APPROACH

Merge them pairwise in rounds: k lists become $k/2$, then $k/4$, until one remains. Each round touches every node once and there are $\log k$ rounds. The alternative is a min-heap of the k current heads, which gives the same $O(N \log k)$ but costs $O(k)$ space.

Complexity Time $O(N \log k)$ where N is the total node count · Space $O(1)$ for pairwise, $O(k)$ for the heap

C++ SOLUTION

```
ListNode* mergeTwoForK(ListNode* a, ListNode* b) {
    ListNode dummy(0);
    ListNode* tail = &dummy;
    while (a && b) {
        if (a->val <= b->val) { tail->next = a; a = a->next; }
        else { tail->next = b; b = b->next; }
        tail = tail->next;
    }
    tail->next = a ? a : b;
    return dummy.next;
}

// Pairwise merging: k -> k/2 -> k/4 -> 1. O(N log k) time, O(1) extra space.
ListNode* mergeKLists(vector<ListNode*>& lists) {
    if (lists.empty()) return nullptr;
    int n = lists.size();
    while (n > 1) {
        int half = (n + 1) / 2;
        for (int i = 0; i < n / 2; i++)
            lists[i] = mergeTwoForK(lists[i], lists[i + half]);
        n = half;
    }
    return lists[0];
}
```

WATCH OUT

Merging one list at a time into an accumulator is $O(N \cdot k)$ and will TLE — that is the trap this problem is built around. Handle the empty vector, and note that individual lists may be null.

26 LC 445 · Add Two Numbers II

MEDIUM Pattern 9 — Arithmetic

PROBLEM

Same as LC 2, but the digits are stored in **forward** order (most significant first), and you may not modify the input lists.

APPROACH

Addition has to start at the least significant digit, which is now at the *tail*. Push both lists onto stacks to get reverse access without reversing them. Build the answer by prepending each new digit, which produces the correct forward order automatically.

Complexity Time $O(n + m)$ · Space $O(n + m)$ for the stacks

C++ SOLUTION

```
// Digits stored MOST significant first, and we may not reverse the input.
// Stacks give us reverse access; building by prepending gives the right order.
ListNode* addTwoNumbersII(ListNode* a, ListNode* b) {
    stack<int> sa, sb;
    for (; a; a = a->next) sa.push(a->val);
    for (; b; b = b->next) sb.push(b->val);

    ListNode* head = nullptr;
    int carry = 0;
    while (!sa.empty() || !sb.empty() || carry) {
        int sum = carry;
        if (!sa.empty()) { sum += sa.top(); sa.pop(); }
        if (!sb.empty()) { sum += sb.top(); sb.pop(); }
        carry = sum / 10;
        head = new ListNode(sum % 10, head); // prepend
    }
    return head;
}
```

WATCH OUT

If reversing the inputs were allowed, you could reuse LC 2 and reverse the result — say so, then give the stack solution as the one that respects the constraint. Prepending with `new ListNode(d, head)` replaces a final reversal pass.

27 LC 430 · Flatten a Multilevel Doubly Linked List

MEDIUM Pattern 12 — Design

PROBLEM

A doubly linked list where nodes may also have a `child` pointer to another such list. Flatten it into a single-level doubly linked list, with each child list spliced in immediately after its parent. All `child` pointers must end up null.

APPROACH

Walk the list. On finding a child, splice the entire child chain in between the current node and its successor: remember `next`, attach the child, walk to the child chain's tail, reattach the remembered `next` there. The outer loop then walks straight into the child chain, so nested levels are handled without any explicit stack or recursion.

Complexity Time $O(n)$ · Space $O(1)$

C++ SOLUTION

```
class MultiNode {
public:
    int val;
    MultiNode *prev, *next, *child;
    MultiNode(int v) : val(v), prev(nullptr), next(nullptr), child(nullptr) {}
};

MultiNode* flatten(MultiNode* head) {
    for (MultiNode* curr = head; curr; curr = curr->next) {
        if (!curr->child) continue;

        MultiNode* nxt = curr->next;    // remember what came after
        MultiNode* child = curr->child;

        curr->child = nullptr;          // splice the child chain in
        curr->next = child;
        child->prev = curr;

        MultiNode* tail = child;        // walk to the end of the child chain
        while (tail->next) tail = tail->next;

        tail->next = nxt;                // reattach the remainder
        if (nxt) nxt->prev = tail;
        // the loop continues INTO the child chain, so nested children are handled
    }
    return head;
}
```

WATCH OUT

Fix the `prev` pointers on both joins — it is a doubly linked list and graders check them. And null out `curr->child`, which is an explicit requirement.

28 LC 147 · Insertion Sort List

MEDIUM Pattern 5 — Merge & sort

PROBLEM

Sort a linked list using insertion sort.

APPROACH

Grow a sorted list hanging off a dummy node. For each node of the input, scan the sorted part from the dummy until you find the insertion point, then splice it in. Save the input's `next` before relinking — that pointer is destroyed by the splice.

Complexity Time $O(n^2)$ · Space $O(1)$

C++ SOLUTION

```
ListNode* insertionSortList(ListNode* head) {  
    ListNode dummy(0); // sorted part hangs off the dummy  
    while (head) {  
        ListNode* nxt = head->next; // save before we relink head  
  
        ListNode* p = &dummy; // find the insertion point  
        while (p->next && p->next->val < head->val) p = p->next;  
  
        head->next = p->next; // splice head in after p  
        p->next = head;  
  
        head = nxt;  
    }  
    return dummy.next;  
}
```

WATCH OUT

Always restart the scan from the dummy, not from where you last inserted — the next input value may be smaller. If asked for the fast sort, this is LC 148 (merge sort) instead.

29 LC 1721 · Swapping Nodes in a Linked List

MEDIUM Pattern 2 — Gap pointers

PROBLEM

Swap the value of the k -th node from the beginning with the value of the k -th node from the end, and return the head.

APPROACH

Walk $k-1$ steps to reach the k -th from the start. Then advance a second pointer from the head in step with a pointer running from that node to the tail — when the leader hits the end, the follower is on the k -th from the end. Swap the two values.

Complexity Time $O(n)$ · Space $O(1)$ · one pass

C++ SOLUTION

```
ListNode* swapNodes(ListNode* head, int k) {
    ListNode* first = head;
    for (int i = 1; i < k; i++) first = first->next;    // k-th from the start

    ListNode* second = head;                          // k-th from the end
    for (ListNode* p = first; p->next; p = p->next) second = second->next;

    swap(first->val, second->val);    // swapping values is allowed here
    return head;
}
```

WATCH OUT

The problem asks only for the *values* to be swapped, so there is no pointer surgery at all — read the statement carefully before writing a much harder node-swap. When k is the middle of an odd-length list, both pointers land on the same node and the swap is a harmless no-op.

30 LC 109 · Convert Sorted List to Binary Search Tree

MEDIUM Pattern Conversion

PROBLEM

Given a linked list sorted in ascending order, build a **height-balanced** binary search tree from it.

APPROACH

The middle element of a sorted sequence is the root that balances it. Find the middle with fast & slow while tracking the node before it, cut there, and recurse on the left part and on everything after the middle.

Complexity Time $O(n \log n)$ · Space $O(\log n)$ — the $O(n)$ alternative builds the tree inorder while walking the list once

C++ SOLUTION

```
struct TreeNode {
    int val;
    TreeNode *left, *right;
    TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
};

// The middle of a sorted list is the root of a height-balanced BST.
TreeNode* sortedListToBST(ListNode* head) {
    if (!head) return nullptr;
    if (!head->next) return new TreeNode(head->val);

    ListNode *slow = head, *fast = head, *prev = nullptr;
    while (fast && fast->next) { // prev tracks the node before slow
        prev = slow;
        slow = slow->next;
        fast = fast->next->next;
    }
    prev->next = nullptr; // cut the left half free

    TreeNode* root = new TreeNode(slow->val);
    root->left = sortedListToBST(head);
    root->right = sortedListToBST(slow->next);
    return root;
}
```

WATCH OUT

You need `prev` to perform the cut — without it you cannot terminate the left half and the recursion never shrinks. Handle the single-node case explicitly, since `prev` is still null there.

31 LC 725 · Split Linked List in Parts

MEDIUM Pattern 1 + splitting

PROBLEM

Split a linked list into k consecutive parts. Part lengths may differ by at most one, earlier parts must be no shorter than later ones, and some parts may be empty. Return an array of the k heads.

APPROACH

Count the length n . Each part gets n / k nodes and the first $n \% k$ parts get one extra. Walk the list cutting off each part in turn.

Complexity Time $O(n + k)$ · Space $O(k)$ for the output

C++ SOLUTION

```
// n nodes into k parts: the first (n % k) parts get one extra node.
vector<ListNode*> splitListToParts(ListNode* head, int k) {
    int n = 0;
    for (ListNode* c = head; c; c = c->next) n++;

    int size = n / k, extra = n % k;
    vector<ListNode*> res(k, nullptr);

    ListNode* curr = head;
    for (int i = 0; i < k && curr; i++) {
        res[i] = curr;
        int len = size + (i < extra ? 1 : 0);
        for (int j = 1; j < len; j++) curr = curr->next; // walk to this part's tail
        ListNode* nxt = curr->next;
        curr->next = nullptr; // cut
        curr = nxt;
    }
    return res;
}
```

WATCH OUT

Initialise the result vector to k nulls so that when $n < k$ the trailing entries are empty lists rather than missing. Cut each part with `curr->next = nullptr` after saving the successor.

32 LC 622 · Design Circular Queue

MEDIUM Pattern 12 — Design

PROBLEM

Implement a fixed-size circular queue supporting `enqueue`, `dequeue`, `Front`, `Rear`, `isEmpty` and `isFull`. `Front` and `Rear` return `-1` when the queue is empty.

APPROACH

A ring buffer over a fixed vector. Store `head` and a `count` rather than head and tail indices — that removes the classic ambiguity where a full queue and an empty queue produce identical index pairs. The rear slot is $(\text{head} + \text{count} - 1) \% \text{cap}$.

Complexity All operations $O(1)$ · Space $O(k)$

C++ SOLUTION

```
// A ring buffer. Storing head + count (instead of head + tail) avoids the
// classic "is it full or empty?" ambiguity entirely.
class MyCircularQueue {
    vector<int> q;
    int head, count, cap;
public:
    MyCircularQueue(int k) : q(k), head(0), count(0), cap(k) {}

    bool enqueue(int value) {
        if (isFull()) return false;
        q[(head + count) % cap] = value;
        count++;
        return true;
    }
    bool dequeue() {
        if (isEmpty()) return false;
        head = (head + 1) % cap;
        count--;
        return true;
    }
    int Front() { return isEmpty() ? -1 : q[head]; }
    int Rear() { return isEmpty() ? -1 : q[(head + count - 1) % cap]; }
    bool isEmpty() { return count == 0; }
    bool isFull() { return count == cap; }
};
```

WATCH OUT

If you insist on head/tail indices instead, you must either waste one slot or keep a separate full flag. The count-based version sidesteps that entirely and is easier to get right under time pressure.

33 LC 460 · LFU Cache

HARD Pattern 12 — Design

PROBLEM

Design a cache with fixed capacity where `get` and `put` run in $O(1)$ average time. When full, evict the **least frequently used** key; if several tie on frequency, evict the least recently used among them.

APPROACH

Three maps. `kv` holds `key → {value, frequency}`. `freqList` holds `frequency → a std::list of keys at that frequency, most recent at the front` — so the tie-break is just the back of the list. `pos` holds `key → its iterator in that list`, making erasure $O(1)$. A `minFreq` variable tracks which bucket to evict from.

Complexity `get` $O(1)$ · `put` $O(1)$ · Space $O(\text{capacity})$

C++ SOLUTION

```
// Evict the least-frequently-used; ties broken by least-recently-used.
// Three maps give O(1): key->{value,freq}, freq->list of keys (MRU at front),
// key->iterator into that list so erasing is O(1).
class LFUCache {
    int cap, minFreq;
    unordered_map<int, pair<int,int>> kv;           // key -> {value, freq}
    unordered_map<int, list<int>>> freqList;       // freq -> keys, MRU first
    unordered_map<int, list<int>::iterator> pos;    // key -> its iterator

    void touch(int key) {                          // bump a key's frequency by one
        int f = kv[key].second;
        freqList[f].erase(pos[key]);
        if (freqList[f].empty()) {
            freqList.erase(f);
            if (minFreq == f) minFreq++; // that bucket is gone, min shifts up
        }
        int nf = f + 1;
        kv[key].second = nf;
        freqList[nf].push_front(key);
        pos[key] = freqList[nf].begin();
    }

public:
    LFUCache(int capacity) : cap(capacity), minFreq(0) {}

    int get(int key) {
        if (cap <= 0 || !kv.count(key)) return -1;
        touch(key);
        return kv[key].first;
    }

    void put(int key, int value) {
        if (cap <= 0) return;
        if (kv.count(key)) { kv[key].first = value; touch(key); return; }

        if ((int)kv.size() == cap) {               // evict LFU, then LRU within it
            int evict = freqList[minFreq].back(); // back = least recently used
            freqList[minFreq].pop_back();
            if (freqList[minFreq].empty()) freqList.erase(minFreq);
            kv.erase(evict);
            pos.erase(evict);
        }
        kv[key] = {value, 1};
        freqList[1].push_front(key);
    }
};
```

```
        pos[key] = freqList[1].begin();  
        minFreq = 1;                      // the new key has frequency 1  
    }  
};
```

WATCH OUT

`minFreq` only ever moves in two places: it becomes 1 whenever a new key is inserted, and it increments when the bucket it points at is emptied by a touch. Reasoning through those two lines is the whole difficulty of this problem. Also handle `capacity == 0`, which LeetCode does test.